

# ADK vs RAG: How to Choose the Right AI Stack

Use a simple test — should the AI act or should it recall — to decide between an agent architecture, a retrieval pipeline, or both.

For developers and technical product people deciding whether to build an AI agent, a retrieval system, or a combination of the two · 27 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

## What matters

- ✓ Agent architectures (ADK-style) are for AI that needs to act: follow steps, call tools, and produce consistent, repeatable behavior.
- ✓ RAG is for AI that needs to know: it retrieves relevant text from your documents before generating an answer, so the answer is grounded in that source instead of the model's internal training.
- ✓ The fastest way to choose is to ask one question: does this system need to act, or does it need to recall?
- ✓ ADK suits multi-step workflows, tool use, and tasks where the value comes from reasoning and repeatable logic, not memory.
- ✓ RAG suits domains where the source of truth is a document set that is large, detailed, or changes often — retrieval keeps answers current without retraining a model.
- ✓ Hybrid systems are the norm in real deployments: the agent layer handles task flow and decisions, while a retrieval layer supplies it with accurate, up-to-date facts.
- ✓ A retrieval step and a tool-calling step are not mutually exclusive — retrieval can itself be exposed to an agent as just another tool it decides to call.

## What it covers

- 01 The Hardware Store Analogy: Two Kinds of AI Systems** 0:00  
A hardware store has a tool aisle for doing work and a reference aisle for knowing how to do it correctly — AI systems split the same way.
- 02 ADK: Designing AI That Acts and Reasons** 0:43  
ADK-style systems give an AI model the ability to reason step by step and call tools to complete multi-step tasks predictably.
- 03 RAG: Designing AI That Retrieves and Knows** 2:06  
RAG connects a model to your documents and retrieves the relevant pieces before it answers, trading the model's internal guesswork for grounded, sourced text.
- 04 Choosing ADK: When Your AI Needs To Do Things** 2:51  
The core test is 'act or recall?' — ADK says 'do something for me,' and it fits anywhere the value comes from reasoning through steps rather than looking something up.
- 05 Choosing RAG: When Your Data Is the Source of Truth** 3:37  
Choose RAG when accuracy has to come directly from documents, especially where content is high-volume, detailed, or constantly changing.
- 06 Hybrid Systems: Reasoning Paired With Domain Knowledge** 4:20

Most real systems use both: an ADK-style agent handles the task flow and decisions, while a RAG component supplies it with grounded facts along the way.

## 07 A Practical Decision Checklist 5:42

Back to the hardware store: use the tool aisle to build, the reference aisle to make sure the build is correct, and pick based on whether the task is to act, to know, or both.

## Glossary

**ADK (Agent Development Kit)** — A term for agent-style architectures built around action and reasoning — the AI follows steps, calls tools, and produces consistent, repeatable behavior.

**Agent** — An AI system that runs in a loop, deciding on each turn whether to call a tool or produce a final answer, rather than generating one response in isolation.

**Embedding** — A numeric vector representation of a piece of text, positioned so that text with similar meaning ends up close together in vector space.

**Grounding** — Basing a model's answer on retrieved source material rather than on patterns memorized during training, so the answer can be traced back to a specific document.

**Triage** — Classifying and routing an incoming request (a ticket, a message) to the right handler or process based on rules or logic.

**RAG (Retrieval-Augmented Generation)** — An architecture where a model retrieves relevant text from a document store before generating its answer, so the answer is grounded in that source material.

**Tool use / function calling** — The mechanism by which a model can invoke an external function or API mid-conversation and receive its result back as new information to reason over.

**Vector database** — A database optimized for storing embeddings and finding the ones closest in meaning to a given query vector.

**ReAct pattern** — An agent loop of reason, then act, then observe the result, then reason again — the mechanical basis of most tool-calling agents.

**Hybrid system** — An architecture where an agent (ADK-style) controls the task flow and calls a retrieval pipeline (RAG) as one of its tools whenever a step needs grounded information.

Explanations are AI-generated. Check anything you intend to rely on.